School of Electronic Engineering and Computer Science

# EBU6304 – Software Engineering Group Project

35% coursework.

# International School Teaching Assistant Recruitment System

## - Software Development Using Agile Methods

## 1. General Information

In the coming weeks, your team will develop a software application that will be used by BUPT International School for recruiting Teaching Assistants. Agile methods should be applied throughout all stages of development, including requirements, analysis and design, implementation, and testing. Iterations (Sprints) should be carefully planned and documented to demonstrate incremental delivery, continuous feedback, and reflective improvement.

There are no restrictions on what may be included in the final product, and the specification provided in this handout contains only high-level, abstract requirements. It is important to recognise that determining software requirements is one of the most critical and complex phases of any development project. You should apply appropriate requirement-finding techniques and Agile practices to identify the actual requirements at a suitable level of detail. Most importantly, you must prioritise the features to be implemented through iterations, considering both feasibility and customer needs. As in real-world software development, you should define the project scope clearly and appropriately. Keep your design simple, modular, and extensible. Justifying your decisions, process, trade-offs, and design choices is an essential part of the coursework.

**Key Dates:**
Handout release date: **6th March 2026**
First assessment: **22nd March 2026**
Intermediate assessment: **12th April 2026**
Final assessment: **24th May 2026**

# 2. Project Specification

## *2.1 Basic Requirements*

BUPT International School recruits Teaching Assistants (TA) each semester to support academic modules and various school activities. Currently, the application process relies on forms and Excel files. A software application is needed to streamline the workflow. You will act as an Agile software development team to develop a simple recruitment system.

The system should address the specific needs of BUPT International School to recruit TAs for Module Organisers (MO) and other activities (such as invigilation). While part of the task requires you to analyse the recruitment process and identify requirements using suitable techniques, here are some suggested functions to assist you in getting started. Suggested features can include the following, but not limited to:

- TA can create an applicant profile
- TA can upload CV
- TA can find available jobs
- TA can apply for jobs
- TA can check application status
- MO can post jobs
- MO can select applicants for jobs
- Admin can check TA's overall workload

Some AI-powered features could include:

- Matching skills between jobs and applicants
- Identifying missing skills for applicants
- Balancing workload

A complete prototype of the application must be produced. While it is not required to implement fully functional code for every feature represented in the prototype, the team must implement a selected set of CORE functions. These core features should demonstrate the system's primary value.

## *2.2 Other Requirements (mandatory)*

- The software must be developed as either
  a) A stand-alone Java application, OR
  b) A lightweight Java Servlet/JSP Web-based application.
- All input and output data should be stored in simple text file formats. For example plain text (.txt), CSV, JSON, or XML. Do not use a database.

**Adhering to the above restrictions is essential to ensure 1: focus remains on fundamental Software Engineering principles, rather than on the complexities of frameworks or languages. Introducing advanced tools, such as Spring Boot or database integration, could divert attention toward framework-specific features and configuration, which are not the primary learning objectives of this module. 2: the system remains compatible with existing data and interoperable with legacy systems. 3: fairness across all students with the necessary prerequisite knowledge and skills.*

Your tasks include defining detailed requirements, designing, developing, and testing the software described above using Agile methodologies. You have the freedom to design the system as you see fit, provided it meets the core customer needs and the project scope is clearly defined.

### *2.3 AI-Assisted Development*

You are encouraged to use AI to assist various aspects of the project, including brainstorming, story writing, prototyping, design, coding, debugging and testing. However, you should not rely solely on AI-generated content, as AI tools have limitations. Outputs may lack domain-specific knowledge, contain subtle logic errors, produce inefficient code, or overlook ethical and privacy considerations. Additionally, AI solutions are often generic and may not reflect the critical thinking and adaptability required for real-world problem-solving. To ensure understanding and avoid over-reliance, you should be prepared to answer questions during live demonstration, such as "Why did AI make this recommendation?" or "How did you modify AI's solution?" to demonstrate your reasoning and modifications.

If you implement AI-powered functions (e.g. skill matching, recommendation), you must integrate AI responsibly and critically. AI-generated outputs must not be blindly accepted. Your system must combine structured logic with AI-based interpretation and provide explainable results.

## 3. Agile Management and Version Control

Each group has 6 students. You are the Agile team working together to complete the project. All students in a group must work on ALL aspects of the project, to obtain full software engineering skills. You should use the techniques you have learnt in the lectures to manage the project, e.g., Scrum, daily standup meetings, working around a table, scrum master and decision making, etc.

Each team member must have a visible branch and corresponding commits on GitHub for their assigned development tasks. **These will serve as key evidence of individual contributions**.

Each group will be assigned a teaching assistant to support, give feedback, monitor the group's progress and individual contribution. Your teaching assistant should be your first point of contact for questions or issues. The teaching assistants will regularly check both your group's progress and individual contributions. Teaching assistants are NOT responsible for marking the project. Marking is done by the lecturers in the Software Engineering teaching team. Make sure your teaching assistant has access to your GitHub repository. Refer to the teaching assistant allocation file on QMPlus to see the contact of your allocated teaching assistant.

All members of group should be collaborators to the main project repository. The main project repository (i.e. the master branch) MUST be accessible to the assigned teaching assistant. The teaching assistant may request to access the sub-branches if necessary.

GitHub should be used explicitly to demonstrate your contributions, through activities such as through commits, forks, pull requests, Issues, and README updates. Each member must actively participate to demonstrate clear engagement and contribution within the team. Insufficient evidence on GitHub will be considered inadequate contribution during marking, with no exceptions.

## 4. Timeline

These are suggested timeline of activities and deliverable for each week. There are 8 scheduled lab sessions in BUPT teaching weeks 3,4,5,6,9,10,11,12, you may use the scheduled lab time for project delivery and personal reflection.

| BUPT week | Scheduled Lab | Activities and Deliverable | Assessment Point | Weight (%) |
|---|---|---|---|---|
| 1 | - | Group and GitHub Repository Setup | - | - |
| 2 | - | Story writing workshop<br>Prototype with user feedback | - | - |
| 3 | 22 March | Iteration 1. | First assessment<br>• Online submission on QM+<br>1. Product Backlog<br>2. Prototype<br>3. A brief report<br>• GitHub | 30% |
| 4 | 29 March | Iteration 1 continues.<br>Working Software version 1. | - | - |
| 5 | 5 April | Iteration 2 | - | - |
| 6 | 12 April | Iteration 2 continues.<br>Working Software version 2. | Intermediate assessment<br>• Demonstration and Viva<br>• GitHub | 20% |
| 7 | - | Iteration 3. | - | - |
| 8 | - | Iteration 3 continues.<br>Working Software version 3. | - | - |
| 9 | 3 May | Iteration 4 | - | - |
| 10 | 10 May | Iteration 4 continues.<br>Working Software version 4. | - | - |
| 11 | 17 May | Preparing for final delivery. | - | - |
| 12 | 24 May | Final Product Delivery | Final assessment<br>• QM+ submission:<br>1. Software<br>2. Demonstration video<br>3. A short report<br>• Demonstration and viva<br>• GitHub | 50% |
| | | | | **100%** |

# 5. Staged Assessment

This coursework will be managed and assessed through three assessment points.

### *5.1. First assessment – 30%*

This involves the product backlog, prototype and a brief report.

**QMPlus Submission:**

- **The product backlog**, an Excel file (a template is provided on QM+).
  Filename: *ProductBacklog_groupXXX.xlsx*, where XXX is your group number.
  It should contain all user stories with acceptance criteria, priority, estimation, and iteration planning.
- **The prototype**, a PDF file.
  Filename: *Prototype_groupXXX.pdf*, where XXX is your group number.
  It should contain the completed prototype. Only a low-fidelity or medium-fidelity prototype is required.
- **The brief report**, a PDF file (a template is provided on QM+).
  Filename: *Report_groupXXX.pdf*, where XXX is your group number.
  This document should describe the fact-finding techniques, iteration planning, prioritisation, and estimation methods used in the project. (maximum 5 pages).
  It should also include supporting materials (not included in the 5-page limit), such as interview questions, survey results, and customer feedback (if applicable).

**Make a .ZIP file contains all above files and submit on QM+. Only the group leader should submit the files on behalf of the whole group.**

**GitHub:** Your GitHub repository will be checked. Make sure it has been set up properly and clear shows members commits.

### *5.2. Intermediate assessment – 20%*

This involves the checking intermediate versions of the software.

**Demo & Viva:** *This simulates your meeting with the client in real-life scenario where you inform the clients of your progress.* All group members need to attend a demonstration and viva session. You will need to present the progress and demonstrate the latest version of the software (ideally with each member speaking for 2 minutes). Each member will be asked 1 or 2 questions during the viva.

**GitHub:** Your GitHub repository will be checked for the up-to-date versions of the software and each member's contribution. Make sure all sub-branch contributions are all merged to the master branch by the checking date.

### *5.3. Final assessment – 50%*

This involves the checking the final software, demonstration and report.

**QMPlus Submission:**

- **The short report,** a PDF file (a template is provided on QM+).
  Filename: *Report_groupXXX.pdf*, where XXX is your group number.
  This report documents the software design strategy, build plan, and testing strategies and techniques, the use of Gen AI etc. It should contain the sections of Group report (maximum 10 pages) and individual statements (no more than 300 words each). More details can be found in the template.(The specific individual work statement in team divis ion of labor shall cover a complete account of all work content undertaken from the initia l phase to the final stage of the entire project.)

- **The software,** a ZIP file.
  Filename: *Software_groupXXX.zip*, where XXX is your group number.
  It should include the following:
  - Source code.
  - Test programs.
  - Code documentation, such as JavaDocs.
  - A user manual including key screenshots of the application (at least one for each main frame)
  - A readme file, providing instructions for setting up, configuring, and running the software.

- **The demonstration video,** record a short video to demonstrate your software and submit it to QM+. Requirements of the demonstration video:
  - Recording can be made using any screen capture software.
  - No more than 10 minutes.
  - You should demonstrate the running of your software, as if you do the demonstration to sell your product. You should demonstrate the operation of all functions. It's recommended to demonstrate error handling as well.
  - You must explain your operations in the video, your voice must be recorded, and the narration must be in English.
  - The video format does not matter, as you will upload it to QM+ Media.
  - The video must not be post-edited, and it must not contain any AI-generated content.

**Demo & Viva:** *This simulates your final meeting with the client in real-life scenario where you deliver the final product and hope to gain user satisfaction.* All group members need to attend a demonstration and viva session. You will be required to complete acceptance testing tasks to verify that the software meets the specified requirements. Each member will also be asked 1 or 2 questions during the viva.

**GitHub:** Your GitHub repository will also be checked for the latest versions of the software and each member's contribution. Make sure all sub-branch contributions are all merged to the master branch by the checking date.

# 6. Marking Criteria

For each assessment stage, **your marks = group marks × individual factor**.

## First assessment:

### Group Marks

| Component | Detail | Max Marks |
|---|---|---|
| **Product Backlog** | The correctness of defining the users and scope | 2 |
| | The correctness of writing user stories | 5 |
| | The completeness of product backlog | 3 |
| **Prototype** | Functionality | 5 |
| | Usability | 5 |
| **Report** | Describe the fact-finding techniques, iteration plan, prioritisation and estimation methods used in the project. | 5 |
| **Evidence** | Sufficient evidence to support the statements in the report. | 5 |
| **Total** | | **30** |

### Individual Factor

| Component | Detail | Factor |
|---|---|---|
| **Contributions** | Regular commits on GitHub | 1/2 |
| | Observations from the teaching assistant | 1/2 |
| **Total** | | **1** |

## Intermediate assessment:

### Group Marks

| Component | Detail | Max Marks |
|---|---|---|
| **Software Version 1 & 2 (GitHub)** | Decent progress has been made for each version. The improvement of each version addresses existing problems. | 5 |
| **Project Management (GitHub)** | The GitHub network shows active engagement from all members with respective branches contributing and merged for the release of the new version. | 5 |
| **Demo & Viva** | Excellent teamwork in presenting the current version of software. All questions are answered clearly. | 10 |
| **Total** | | **20** |

### Individual Factor

| Component | Detail | Factor |
|---|---|---|
| **Demo & Viva** | Presentation | 1/2 |
| | Q&A | 1/2 |
| **Total** | | **1** |

## Final assessment:

### Group Marks

| Component | Detail | Max Marks |
|---|---|---|
| **Demonstration video and software** | Design | 10 |
| | Implementation | 10 |
| | Testing | 5 |
| **Short report** | Content quality | 10 |
| **Project Management (GitHub)** | The GitHub network shows active engagement from all members with respective branches contributing and merged for the release of the new version. | 5 |
| **Demo & Viva** | Pass all acceptance testing tasks. All questions are answered clearly. | 10 |
| **Total** | | **50*** |

### Individual Factor

| Component | Detail | Check |
|---|---|---|
| **Contribution** | Contribution on GitHub | 1/4 |
| | Reflection in the Report | 1/4 |
| **Demo & Viva** | Testing task | 1/4 |
| | Q&A | 1/4 |
| **Total** | | **1** |

*** A student who demonstrates significant contribution and leadership in the group may receive up to 10% additional marks added to the group's final assessment mark.**